

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **2**
Topic : **5**

Bigdata distribution models and Map Reduce

Objectives

This session will give the knowledge about

- Using NoSQL to manage big data
- What is a big data NoSQL solution?
- Bigdata distribution models and Map Reduce
- Classifying big data problems
- The challenges of distributed computing for big data

Using NoSQL to manage big data

Twenty years ago, companies managed datasets that contained approximately a million internal sales transactions, stored on a single processor in a relational database.

As organizations generated more data from internal and external sources, datasets expanded to billions and trillions of items.

The amount of data made it difficult for organizations to continue to use a single system to process this data. They had to learn how to distribute the tasks among many processors. This is what is known as a big data problem.

What is a big data NoSQL solution?

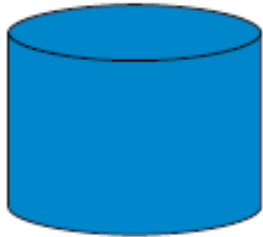
A big data problem is any business problem that's so large that it can't be easily managed using a single processor.

Big data problems force you to move away from a single-processor environment toward the more complex world of distributed computing.

Here are some of the challenges you face when you move from a single processor to a distributed computing system.

What is a big data NoSQL solution?

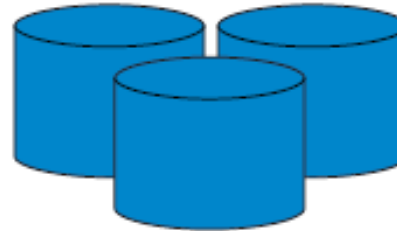
One database



- Easy to understand
- Easy to set up and configure
- Easy to administer
- Single source of truth
- Limited scalability

OR

Many databases



- Data partitioning
- Replication
- Clustering
- Query distribution
- Load balancing
- Consistency/Syncing
- Latency/Concurrency
- Clock synchronization
- Network bottlenecks/failures
- Multiple data centers
- Distributed backup
- Node failure
- Voting algorithms for error detection
- Administration of many systems
- Monitoring
- Scalable if designed correctly

Big data use cases

Here are some typical big data use cases:

- Bulk image processing
- Public web page data
- Remote sensor data
- Event log data
- Mobile phone data
- Social media data
- Game data
- Open linked data

Big data use cases

Among these use cases, some problems can be described as independent parallel transforms since the output from one transform isn't used as an input to another.

Read and write to key-value stores or distributed file systems like **Amazon's Simple Storage Service (S3)** or **Hadoop Distributed File System (HDFS)** and may not need the advanced features of a document store or an RDBMS.

Big data problems like event log data and game data do need to store their data directly into structures that can be queried and analyzed, so they will need different NoSQL solutions

Good NoSQL Solutions to Big data problems

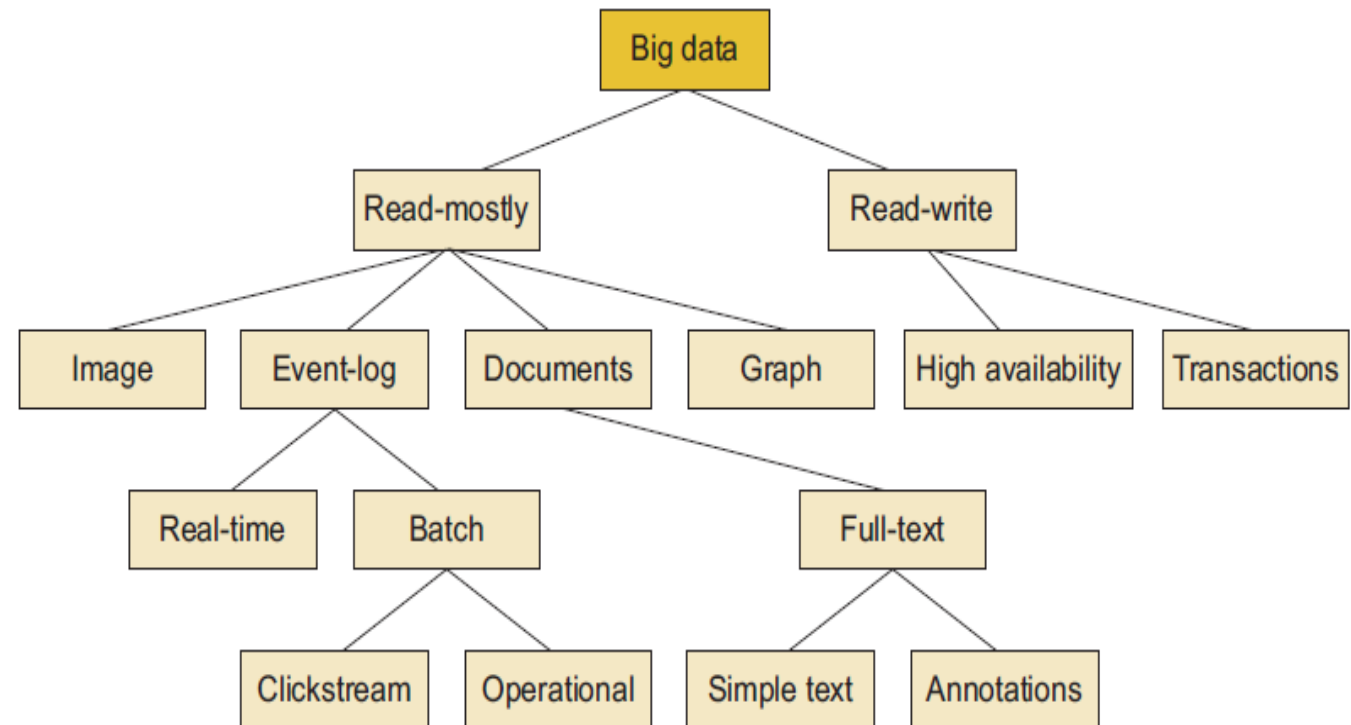
To be a good candidate for a general class of big data problems, **NoSQL solutions should**

- Be efficient with input and output and scale linearly with growing data size.
- Be operationally efficient. Organizations can't afford to hire many people to run the servers.
- Require that reports and analyses be performed by nonprogrammers using simple tools.
- Meet the challenges of distributed computing, including consideration of latency between systems and eventual node failures.

Understanding the types of big data problems

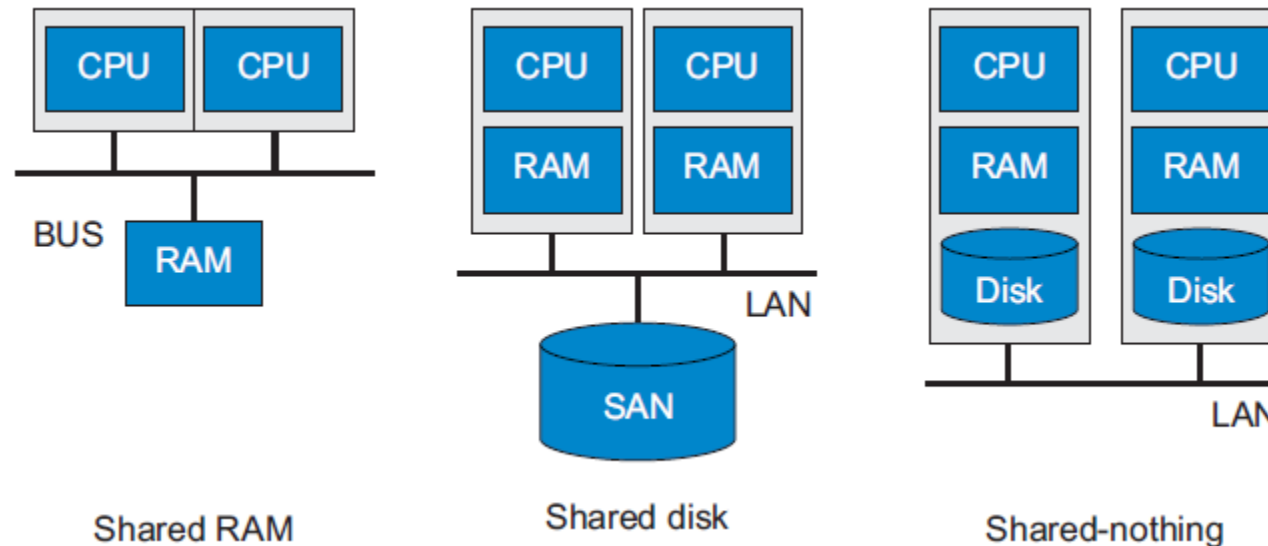
There are many **types of big data problems**, each requiring a different combination of NoSQL systems.

A Good example of a high-level big data classification system is



Analyzing big data with a shared-nothing architecture

There are **three ways** that resources can be shared between computer **systems**: shared RAM, shared disk, and shared-nothing.



Shared-nothing architecture

The left panel shows a shared RAM architecture, where many CPUs access a single shared RAM over a high-speed bus. This system is ideal for large graph traversal.

The middle panel shows a shared disk system, where processors have independent RAM but share disk using a storage area network (SAN).

The right panel shows an architecture used in big data solutions: cache-friendly, using low-cost commodity hardware, and a shared-nothing architecture.

Choosing distribution models: Master-Slave VS Peer-to-Peer

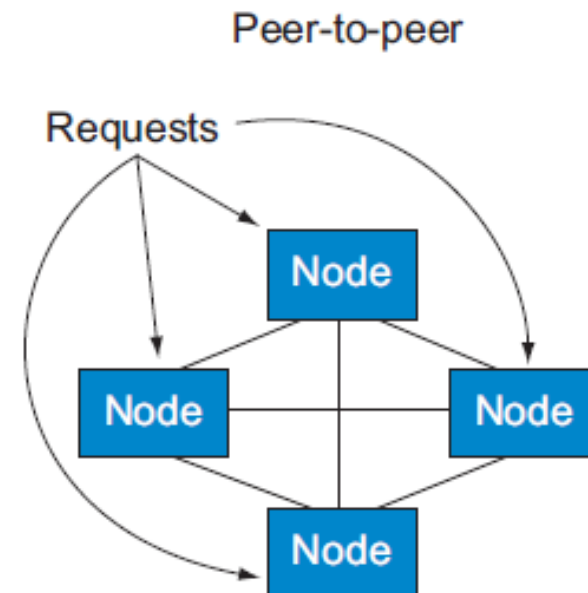
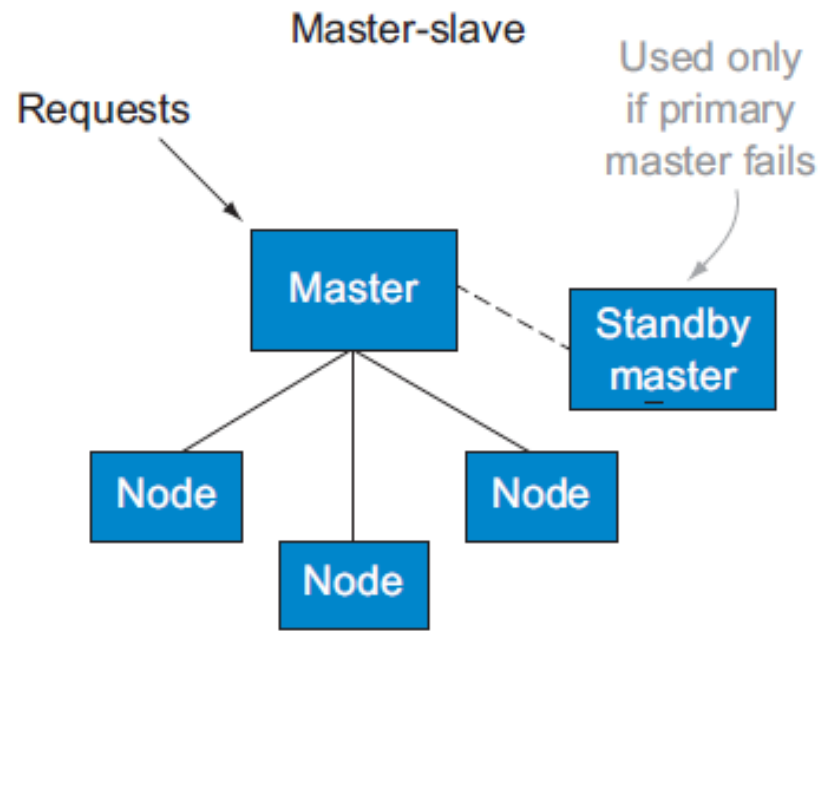
From a distribution perspective, there are two main models: **master-slave and peer-to peer**.

Distribution models **determine the responsibility for processing data when a request is made**.

Understanding the pros and cons of each distribution model is important when you're looking at a potential big data solution.

Peer-to-peer models may be more resilient to failure than master-slave models. Some master-slave distribution models have single points of failure that might impact your system availability.

Master-Slave VS Peer-to-Peer



Master-Slave VS Peer-to-Peer

The panel on the left illustrates a master-slave configuration where all incoming database requests (reads or writes) are sent to a single master node and redistributed from there.

The master node is called the Name Node in Hadoop. This node keeps a database of all the other nodes in the cluster and the rules for distributing requests to each node.

The panel on the right shows how the peer-to-peer model stores all the information about the cluster on each node in the cluster.

If any node crashes, the other nodes can take over and processing can continue.

MapReduce to transform data over distributed systems

MapReduce systems can be used to process large datasets on multiple processors.

If you've been moving data between SQL systems, you're familiar with the **extract, load, and transform (ETL)** process.

SQL-based ETL systems usually **don't have the inherent ability to use a large number of processors** to do their work.

This single-processor bottleneck is common in data warehouse systems as well as areas of big data.

MapReduce

To solve this type of problem, organizations have moved to a distributed transformation model built around the map and reduce functions.

To effectively distribute work evenly over a cluster of processors, the output of a map phase must be a set of key-value pairs where part of the key structure is used to correlate results into the reduce phase.

These functions are designed to be inherently linearly scalable using a large number of shared-nothing processors.

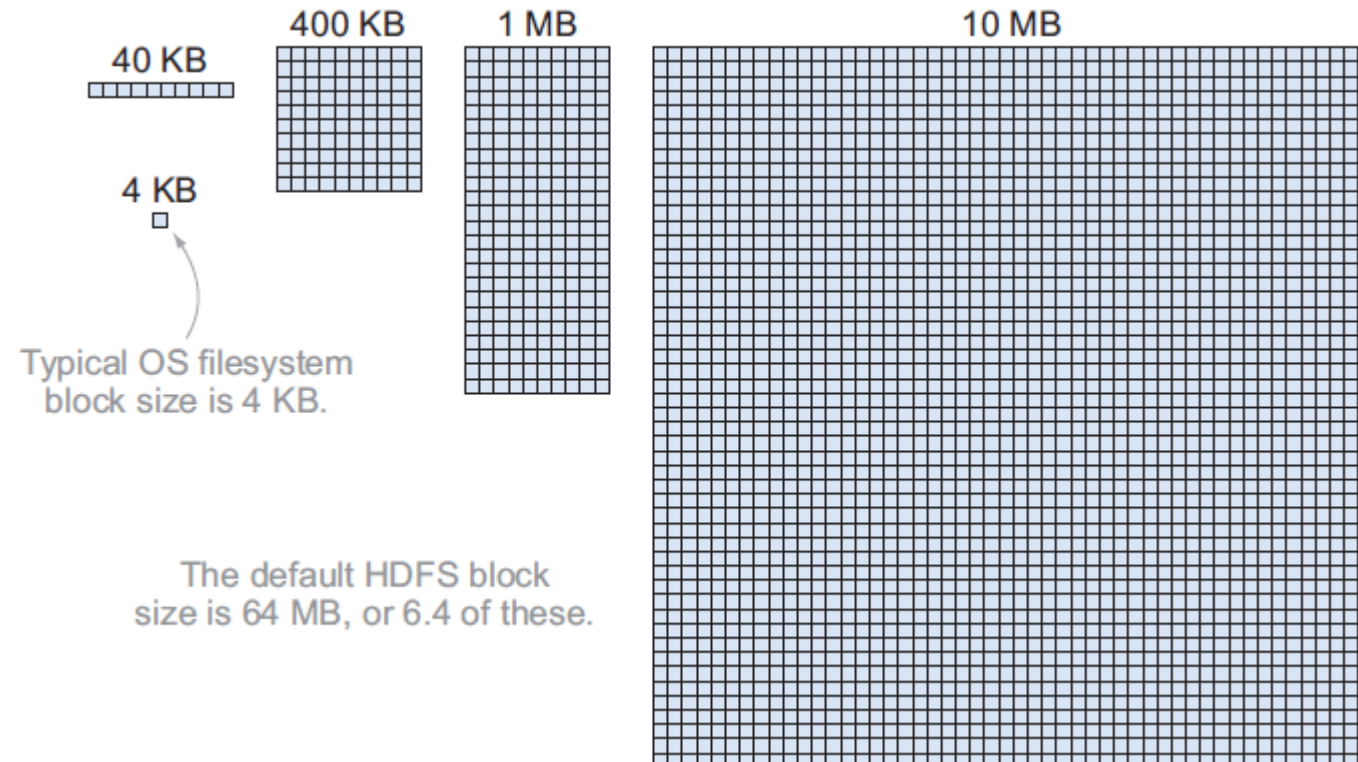
MapReduce

Traditional file systems store data in a single location; if a drive fails, data is restored from a backup drive.

By default, files in HDFS are stored in three locations; if a drive fails, the data is automatically replicated to another drive. It's possible to get the same functionality using a fault-tolerant system like RAID drives.

HDFSs are different: they use a large (64 megabytes by default) block size to handle data. Figure shows how large HDFS blocks are compared to a typical operating system.

MapReduce



MapReduce

MapReduce is a programming framework that allows us to perform distributed and parallel processing on large data sets in a distributed environment.

MapReduce consists of two distinct tasks — Map and Reduce.

As the name MapReduce suggests, reducer phase takes place after the mapper phase has been completed. So, the first is the map job, where a block of data is read and processed to produce key-value pairs as intermediate outputs.

MapReduce

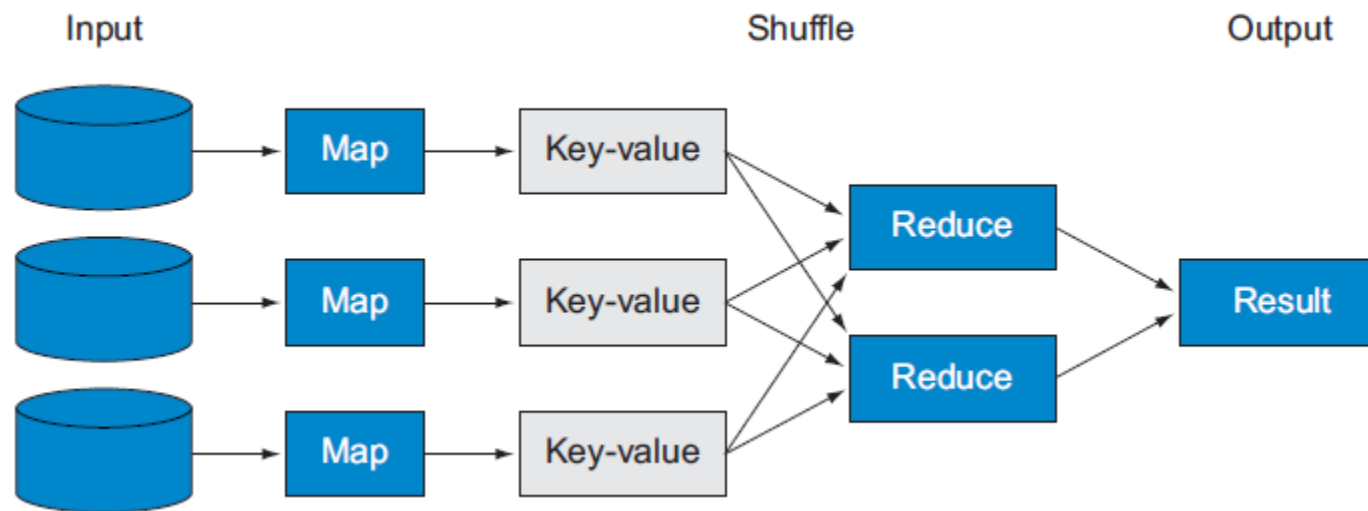
The output of a Mapper or map job (key-value pairs) is input to the Reducer.

The reducer receives the key-value pair from multiple map jobs.

Then, the reducer aggregates those intermediate data tuples (intermediate key-value pair) into a smaller set of tuples or key-value pairs which is the final output.

MapReduce

This figure explain how MapReduce transfers the data over distributed system.



MapReduce

The basics of how the map and reduce functions work together to gain linear scalability over big data transforms.

The map operation takes input data and creates a uniform set of key-value pairs.

In the shuffle phase, which is done automatically by the MapReduce framework, key-value pairs are automatically distributed to the correct reduce node based on the value of the key.

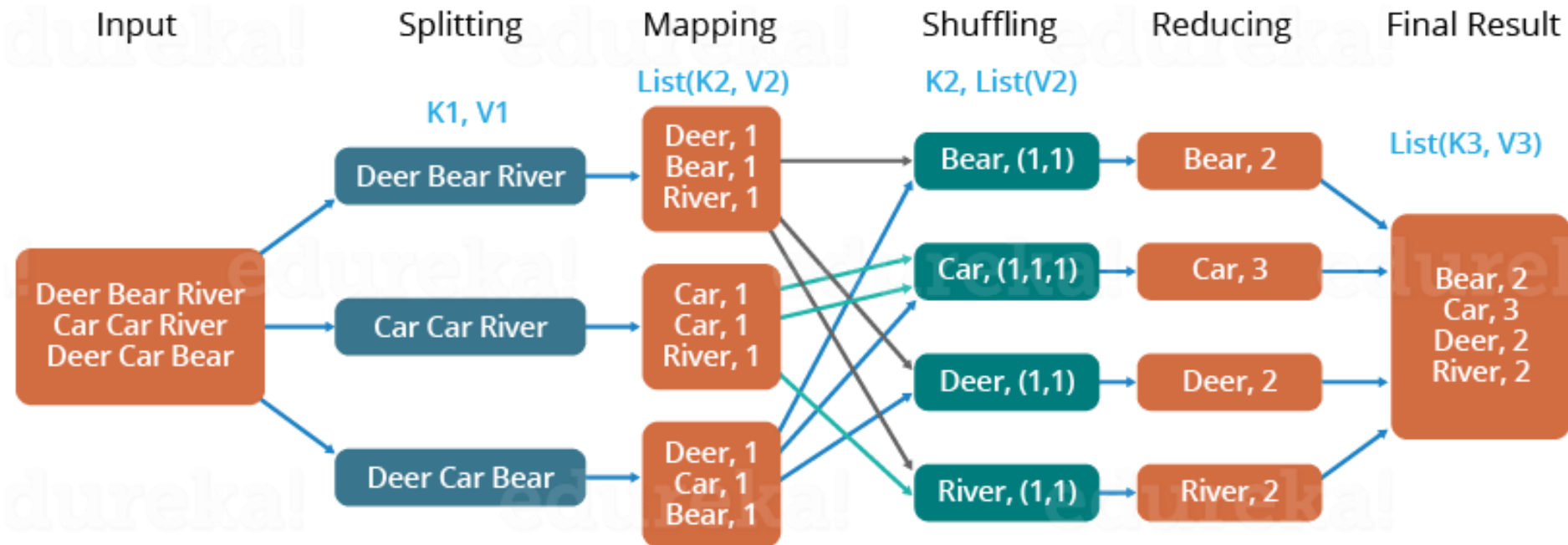
The reduce operation takes the key-value pairs and returns consolidated values for each key. It's the job of the MapReduce framework to get the right keys to the right reduce nodes.

Word Count Example of MapReduce

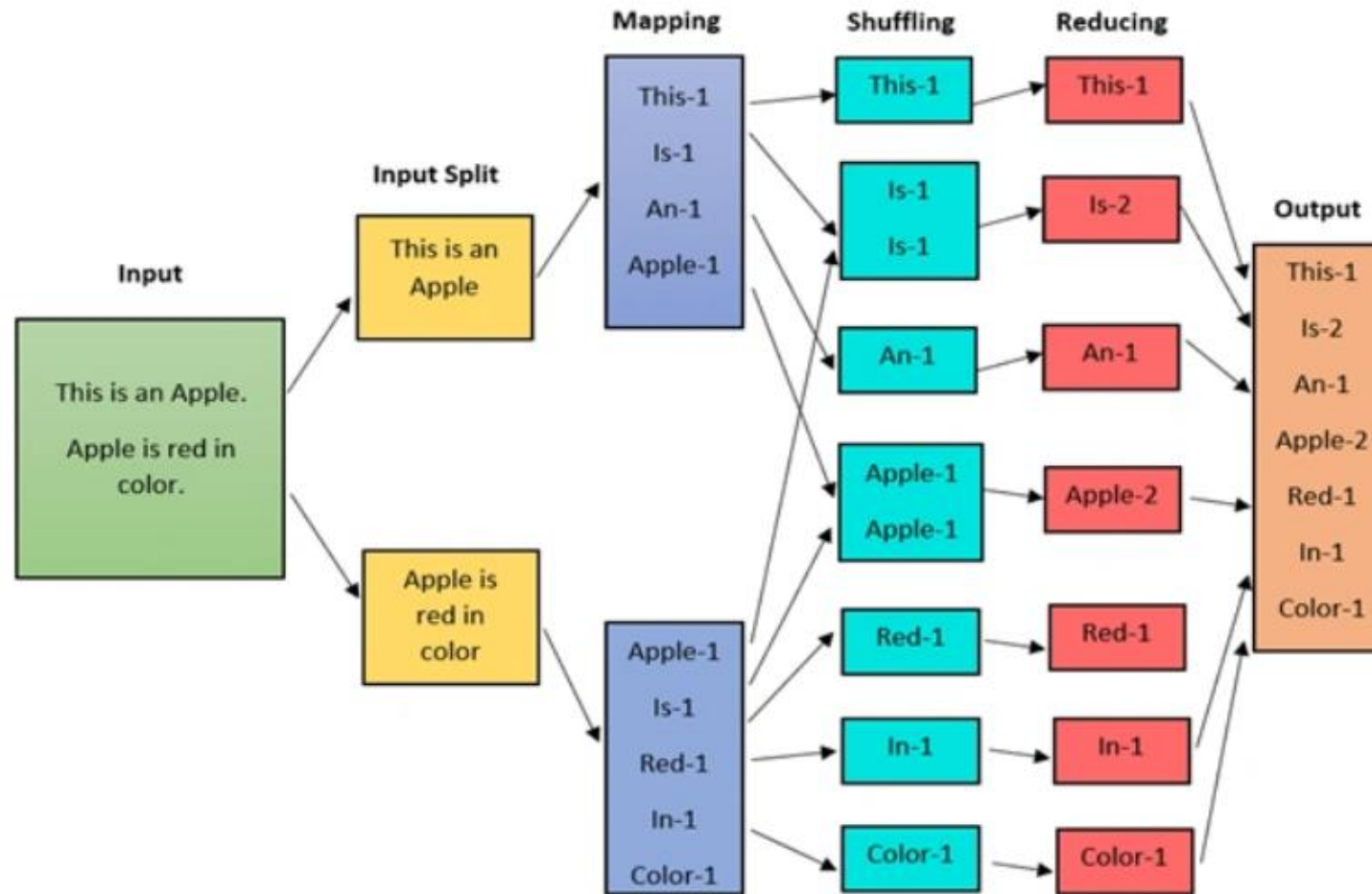
example.txt: Dear, Bear, River, Car, Car, River, Deer, Car and Bear

The Overall MapReduce Word Count Process

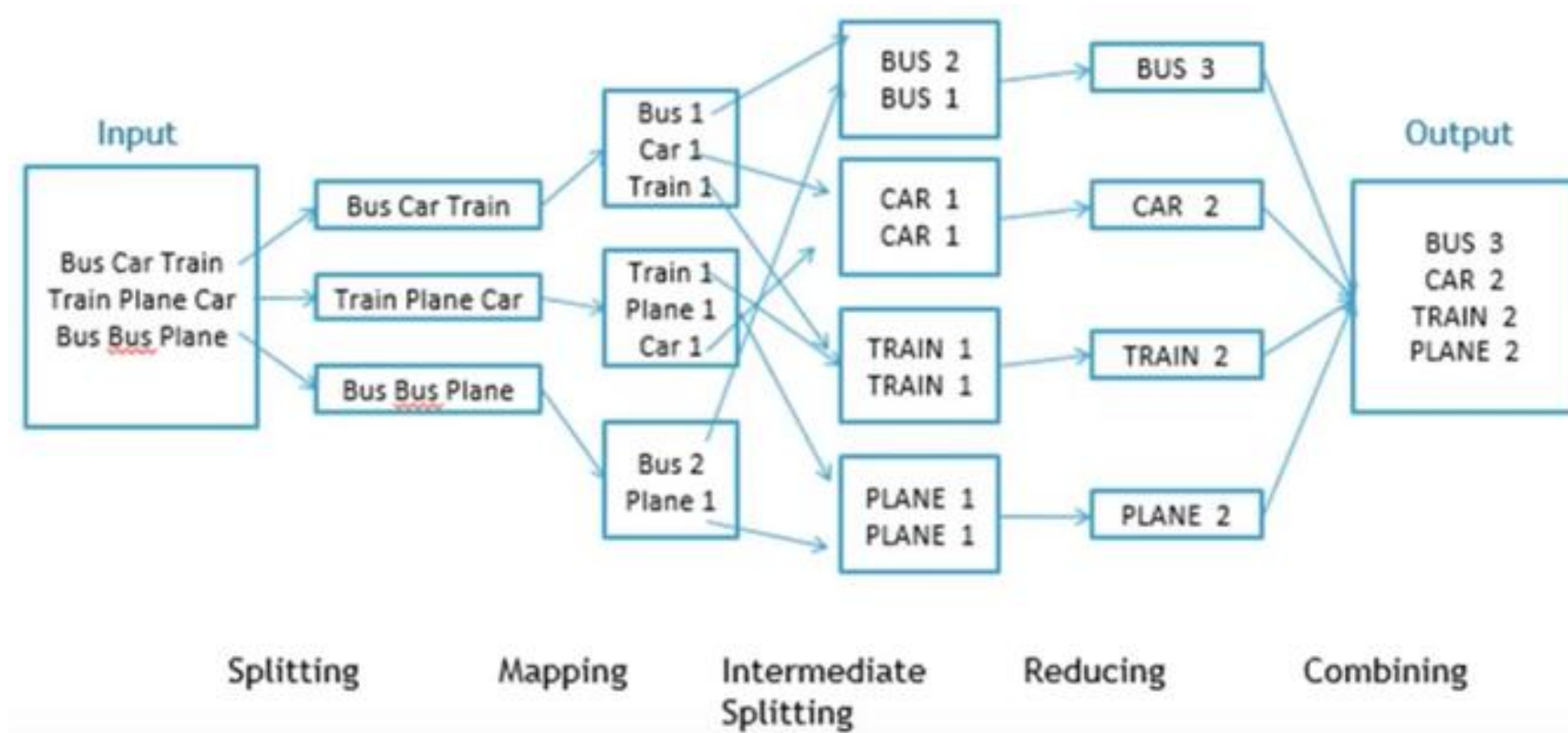
edureka!



Word Count Example of MapReduce



Word Count Example of MapReduce



MapReduce with MongoDB

The basics of how the map and reduce functions work together to gain linear scalability over big data transforms.

The map operation takes input data and creates a uniform set of key-value pairs.

In the shuffle phase, which is done automatically by the MapReduce framework, key-value pairs are automatically distributed to the correct reduce node based on the value of the key.

The reduce operation takes the key-value pairs and returns consolidated values for each key. It's the job of the MapReduce framework to get the right keys to the right reduce nodes.

Summary

This session will give the knowledge about

- Using NoSQL to manage big data
- What is a big data NoSQL solution?
- Bigdata distribution models and Map Reduce
- Classifying big data problems
- The challenges of distributed computing for big data